Compression of a From-Scratch Small Language Model

August 27, 2026

Elena Mannoni [1] **Giovanni Adelfio** [1]

Abstract

This report describes the implementation, training, and compression of a 6.8-million parameter Small Language Model (SLM) built for resource-constrained inference. We implemented a causal Transformer architecture from first principles in PyTorch Lightning (Falcon and The PyTorch Lightning Team, 2019), rather than relying on pre-trained architectures (Wolf et al., 2020). Instead of training directly on the target task, we used a two-stage curriculum: pre-training on *TinyStories* (Eldan and Li, 2023) for general English syntax and short-range coherence, followed by fine-tuning on a structured Question-Answering riddle dataset. An earlier attempt to fine-tune directly on unconstrained multi-turn dialogue (DailyDialog, Li et al., 2017) caused autoregressive instability at this parameter scale, motivating the staged curriculum. We then applied a compression pipeline: structured pruning targeting 30% of attention heads and FFN neurons via L2-norm scoring (Molchanov et al., 2017), reducing the model to 5.9 M parameters and 22.1 MB; and multi-format Post-Training Quantization (PTQ), including INT8 Dynamic (Jacob et al., 2018), an SNR-guided INT8 Hybrid configuration, and 4-bit NormalFloat (NF4) (Dettmers et al., 2023). Low-bitwidth quantization preserved semantic quality, reaching a footprint of 10–12 MB with no significant change in BERTScore (Zhang et al., 2020).

1. Introduction

Scaling Large Language Models (LLMs) to hundreds of billions of parameters creates deployment challenges on edge and consumer hardware (Kaplan et al., 2020; Brown et al., 2020). Many applications require only narrow, specific tasks, for which a small fraction of a frontier model's capacity suffices (Eldan and Li, 2023; Gunasekar et al., 2023). Small Language Models (SLMs) are a computationally viable alternative, provided their training is constrained enough to avoid autoregressive collapse on high-entropy text. This project builds an ultra-lightweight SLM from scratch, including tokenizer, architecture, training curriculum, and compression pipeline. A custom causal Transformer (Vaswani et al., 2017) is first pre-trained on TinyStories for basic grammar and narrative competence, then fine-tuned on a riddles dataset (Hypersniper, 2024) for a single generative task, followed by structured pruning and low-bitwidth quantization (Jacob et al., 2018; Frantar et al., 2022). By formalizing sparsification of the Multi-Head Attention (MHA) and Feed-Forward blocks, we evaluate the limits of memory reduction relative to generative coherence. Section 2 surveys related work. Section 3 describes the training curriculum, tokenizer, and architecture. Section 4 presents the compression pipeline. Section 5 reports results, and Section 6 concludes.

2. Related Works

2.1. Efficient and Small Language Models

The Transformer (Vaswani et al., 2017) underlies most modern autoregressive language models, from GPT-2 (Radford et al., 2019) to large-scale foundation models (Brown et al., 2020; Touvron et al., 2023). Training such models typically requires web-scale corpora and large compute budgets. *TinyStories* (Eldan and Li, 2023) showed that small Transformers can produce fluent, coherent text when trained on a synthetically simplified corpus with restricted vocabulary and grammar, acting as a curriculum that separates linguistic competence from world knowledge. Related work on "textbook-quality" data (Gunasekar et al., 2023) similarly shows curated, low-entropy data can substitute for scale. Our curriculum builds on this: general narrative competence is acquired via TinyStories before task-specific specialization.

[*]Equal contribution [1]Department of Mathematics, Sapienza University of Rome Email: Giovanni Adelfio <adelfio.2151753@studenti.uniroma1.it>, Elena Mannoni <mannoni.2126822@studenti.uniroma1.it>.

Machine Learning 2025, Sapienza University of Rome, 2nd semester a.y. 2024/2025.

2.2. Tokenization

Byte-Pair Encoding (BPE) (Sennrich et al., 2016) is the dominant sub-word tokenization scheme for autoregressive language models. We use the `tokenizers` library's byte-level BPE implementation, in the same family as GPT-2's tokenizer (Radford et al., 2019).

2.3. Pruning

Model compression is broadly split into pruning and quantization. Pruning assumes neural over-parameterization (Frankle and Carbin, 2019). Unstructured magnitude pruning zeros individual weights and requires sparse-matrix support; structured pruning removes entire neurons, channels, or attention heads, giving direct latency and memory gains on standard dense hardware. Importance criteria range from weight-magnitude/L2-norm scores (Han et al., 2015) to gradient- and Taylor-expansion-based saliency (Molchanov et al., 2017), extended to attention heads (Michel et al., 2019; Voita et al., 2019). We use L2-norm scoring, applied to both attention heads and FFN neurons.

2.4. Quantization

Post-Training Quantization (PTQ) maps weights to lower precision (Jacob et al., 2018); Dynamic INT8 converts weights offline and quantizes activations at inference. Below 8 bits, LLM.int8() (Dettmers et al., 2022) introduced a vector-wise 8-bit scheme that isolates outlier feature dimensions to preserve accuracy. Building on this quantization line, QLoRA (Dettmers et al., 2023) then introduced 4-bit NormalFloat (NF4), which represents weights in bins optimized for the roughly Gaussian distributions typical of trained networks. GPTQ (Frantar et al., 2022) and AWQ (Lin et al., 2023) refine per-layer quantization error via second-order or activation-statistics calibration; our SNR-guided hybrid scheme is a lightweight analogue of this family. Distillation (Hinton et al., 2015; Sanh et al., 2019) and low-bit BERT quantization (Zafrir et al., 2019) are complementary axes not explored here.

2.5. Evaluation of Compressed Generative Models

Cross-Entropy loss or Perplexity alone cannot capture high-level semantic coherence. BERTScore (Zhang et al., 2020) compares contextual embeddings of generated and reference text, serving as a proxy for whether quantization noise disrupts the model's reasoning structure, complementing perplexity.

3. Methodology

3.1. Training Curriculum

A central choice of this project is a two-stage curriculum rather than training directly on the target riddle task. **Stage 0 – discarded dialogue attempt.** We first tried fine-tuning directly on unconstrained multi-turn dialogue, using the "Ordinary Life" subset of DailyDialog (Li et al., 2017). This high-entropy setting caused semantic drift and repetitive, degenerate completions at this parameter scale, confirming that a sub-10M-parameter model cannot jointly learn open-domain grammar and dialogue policy from a small corpus. We abandoned this as a primary training source. **Stage 1 – pre-training on TinyStories.** To establish basic grammar, sentence structure, and short-range narrative coherence before any task-specific format, we pre-trained on 50,000 stories from *TinyStories* (Eldan and Li, 2023), streamed from the Hugging Face `datasets` hub. Its restricted vocabulary and simple grammar make it a suitable bootstrapping corpus, separating syntax acquisition from downstream skill acquisition. **Stage 2 – fine-tuning on riddles.** The model was then fine-tuned on `Hypersniper/riddles_v1` (Hypersniper, 2024), restructured into a prompt-answer template (e.g. `"A riddle about {answer}:"` followed by the riddle text) to constrain vocabulary and output structure. An extended variant adds an explanatory field, yielding `RIDDLE: ... ANSWER: ... WHY: ...`. Fine-tuning proceeded from the Stage 1 checkpoint with a reduced learning rate (5×10^{-5}, further annealed later) to avoid overwriting the linguistic priors learned in Stage 1, following standard staged fine-tuning practice (Howard and Ruder, 2018). Training continued cumulatively for about 40 epochs, with early stopping on validation loss (patience 3) selecting the final checkpoint prior to compression. This curriculum mirrors the standard autoregressive pre-train/fine-tune paradigm (Radford et al., 2019; Howard and Ruder, 2018), applied at a scale where the pre-training corpus choice strongly affects training stability.

3.2. Data Curation and Tokenization

We trained a single BPE tokenizer (Sennrich et al., 2016) jointly over the combined TinyStories and riddles corpora, so both stages share one vocabulary with no re-tokenization needed between them. The tokenizer uses a `ByteLevel` pre-tokenizer and decoder (Radford et al., 2019), with a vocabulary size of 4,000 tokens. Special tokens (`[BOS]`, `[EOS]`, `[PAD]`) and control tokens `RIDDLE:`/`ANSWER:` enforce structural boundaries. This small vocabulary reduces the embedding layer's parameter load relative to a typical 32k–50k vocabulary. Data was split 70%/20%/10% (train/val/test) with a fixed seed (42), and sequences were chunked to a context window of 128 tokens with batch size

32 for both stages.

3.3. Custom SLM Architecture

The model is a causal Transformer decoder built in PyTorch Lightning (`GPTLightningModule`) (Falcon and The PyTorch Lightning Team, 2019; Vaswani et al., 2017), with embedding dimension 256, 8 attention heads, and 6 Transformer blocks over a 128-token context. Weights were initialized from $\mathcal{N}(0, 0.02^2)$ and biases set to zero, following GPT-style conventions (Radford et al., 2019). Multi-Head Attention (MHA) computes scaled dot-product attention over a causal mask M (via `torch.triu`):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

where $d_k = 32$. Outputs are normalized and passed to a Feed-Forward Network (FFN) with ReLU activation:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

The original FFN hidden dimension is $4 \times d_{model} = 1024$. Dropout (Srivastava et al., 2014) ($p = 0.1$) is applied after the token and positional embeddings, inside the attention projections, and within the FFN bottleneck. The full architecture, including a separate (untied) language-model head, totals approximately 6.8 M parameters.

3.4. Training Configuration and Optimization

To optimize memory footprint and execution speed, the entire training pipeline was executed natively in 16-bit half precision (FP16) (Micikevicius et al., 2018). We used AdamW (Loshchilov and Hutter, 2019) with a base learning rate of 1×10^{-3} during pre-training and betas $(0.9, 0.99)$. Weight decay (0.1) applied only to 2D weight matrices; LayerNorm parameters and biases were exempted. Learning rate followed Cosine Annealing (`CosineAnnealingLR`, Loshchilov and Hutter, 2017) with η_{min} set to 10% of the initial rate, and early stopping on validation loss (patience 3). During riddle fine-tuning, the learning rate was reduced (to 5×10^{-5} in later runs) to preserve linguistic representations from pre-training while adapting to the riddle format.

4. Compression Framework

4.1. Structured Pruning via L2-Norm Scoring

We applied a two-pronged structured pruning strategy to the fine-tuned checkpoint, targeting 30% of attention heads and 30% of FFN neurons. Because the two components are pruned differently, their effects on the parameter count are distinct. **MHA head zeroing.** For MHA layers, we scored each attention head's importance via a forward hook capturing the L2-norm of its output over a calibration batch:

$$s_{head,h} = \frac{1}{B \cdot T} \sum_{b=1}^{B} \sum_{t=1}^{T} \|O_{b,t,h}\|_2$$

The bottom 30% of heads globally were *functionally disabled* by zeroing the corresponding rows in the Value slice of the input projection (`in_proj_weight`) and columns in the output projection (`out_proj.weight`), while leaving Query and Key projections intact. Because the tensor shapes are preserved, this step does not reduce the stored parameter count; it eliminates the computational contribution of the pruned heads while keeping the residual stream and embedding dimension unchanged. **FFN structural pruning.** For FFN layers, we shrunk the hidden dimension from 1024 to 716 neurons using the L2-norm of the incoming W_1 rows:

$$s_{FFN,i} = \|W_{1,i*}\|_2$$

The lowest-norm 30% of neurons and their corresponding W_2 columns were removed, and the layers were reinstantiated at the smaller size, so savings are realized immediately on dense hardware without sparse-matrix kernels. **Net effect.** The FFN structural pruning is the sole source of parameter reduction. It removed approximately 0.95 M parameters across all 6 blocks, bringing the model from 6.8 M to 5.9 M parameters (≈14% total reduction) and a serialized size of 22.1 MB. A recovery fine-tuning phase (learning rate 1×10^{-4}, dropout $p = 0.2$, two epochs) restored cross-entropy alignment lost during pruning, following the standard prune-then-fine-tune recipe (Han et al., 2015; Frankle and Carbin, 2019). Prior to pruning, we profiled the unpruned model's CPU inference cost with `torch.profiler`, confirming that Linear and MultiheadAttention modules – the targets of our pruning and quantization – dominate both parameter count and inference compute.

4.2. Post-Training Quantization (PTQ) and Sensitivity Profiling

The 5.9M-parameter model was then subjected to multi-format PTQ. **FP16 (Baseline):** Because the model was trained natively in half precision, the FP16 weights served directly as the unquantized baseline for compression evaluation, requiring no further post-training conversion. **INT8 Dynamic:** Weights of all `nn.Linear` modules were mapped to 8-bit integers via `torch.quantization.quantize_dynamic` (Jacob et al., 2018), with activations quantized dynamically at inference. **INT8 Hybrid:** Since uniform quantization can destabilize sensitive layers (Dettmers et al., 2022), we ran a sensitivity profiling pass related to per-layer calibration in GPTQ (Frantar et al., 2022) and AWQ (Lin et al., 2023).

For each block, we computed the Signal-to-Noise Ratio (SNR) between FP16 weights and their INT8 counterparts:

$$\text{SNR}_{layer} = 10 \log_{10} \left(\frac{\sum W^2}{\sum (W - W_{quant})^2} \right)$$

We also measured the validation Perplexity delta (ΔPPL) from quantizing each layer individually. Layers with SNR $<$ 25.0 dB or ΔPPL $>$ 5.0 were kept in FP16; layers with SNR $>$ 35.0 dB and ΔPPL $<$ 0.5 were compressed to INT8, giving a mixed-precision model where layers near the embedding and output heads typically remain full precision. **INT4 (NF4):** Linear layers were mapped to 4-bit NormalFloat via `bitsandbytes` (Dettmers et al., 2023) and `torchao`, optimizing quantization bins for zero-centered normal weight distributions. `nn.MultiheadAttention` modules were left unquantized, as their fused internal projections are not compatible with the 4-bit linear-layer replacement used.

5. Experimental Results and Discussion

Configurations were evaluated on serialized model size and semantic quality via BERTScore (Precision, Recall, F1) (Zhang et al., 2020), alongside validation Perplexity. The pre-pruned baseline recorded a BERTScore F1 of 0.872. After pruning and recovery fine-tuning, the 5.9M-parameter model reached F1 = 0.8698, indicating the two-epoch recovery was sufficient to offset the capacity reduction. Quantization results are in Table 1. All configurations share the same 5.9M-parameter post-pruning architecture and differ only in linear-layer weight precision. INT8 Dynamic reduced footprint by 54.4% relative to the FP16 baseline, reaching 10.09 MB. INT8 Hybrid, despite keeping sensitive layers in FP16, still reduced footprint substantially relative to the 22.10 MB post-pruning checkpoint, though at 13.98 MB it is the largest of the quantized variants due to retained FP16 layers. INT4 reached 11.82 MB, slightly above pure INT8 Dynamic due to the sub-byte unpacking structures and block-wise scale/zero-point statistics required by NF4 (Dettmers et al., 2023). Quantization did not degrade downstream performance: F1 variations (e.g. the increase to 0.8751 for INT4) fall within expected statistical fluctuation for the BERT surrogate (Zhang et al., 2020). With only two calibration batches, however, these differences are not necessarily significant, and a larger held-out set would be needed to draw stronger conclusions.

Table 1. Model Performance Across Quantization Formats (Post-Pruning, 5.9M parameters). Size refers to the serialized model checkpoint on disk.

Model Config	Size (MB)	F1 (Mean)	Precision Format
FP16 (baseline)	22.10	0.8698	16-bit float
INT8 Hybrid	13.98	0.8719	Mixed FP16/INT8
INT8 Dynamic	10.09	0.8742	8-bit integer
INT4 (NF4)	11.82	0.8751	4-bit NormalFloat

6. Conclusions

This study shows that a custom SLM can be heavily compressed without catastrophic degradation. Pre-training on TinyStories (Eldan and Li, 2023) before fine-tuning on a vocabulary-constrained riddles dataset (Hypersniper, 2024), with a jointly-fit BPE tokenizer, stabilized a 6.8 M-parameter causal model after an earlier dialogue-only attempt had failed. Structured pruning reduced the model to 5.9 M parameters by structurally shrinking the FFN matrices and functionally zeroing redundant attention heads. Subsequent PTQ, notably INT8 Dynamic and INT4 (NF4), reduced model size by up to 54% relative to the FP16 baseline, to 10–12 MB. SNR-based sensitivity profiling enabled layer-wise precision control in the hybrid INT8 configuration. Preserved BERTScore alignment indicates SLMs can tolerate aggressive low-bitwidth mapping, supporting deployment of specialized NLP tasks on constrained edge processors.

6.1. Limitations and Future Work

Our BERTScore evaluation used few calibration batches; a larger held-out riddle test set would strengthen the reliability of F1 differences across formats. We did not benchmark on-device inference latency or energy consumption after compression; the CPU profiling performed before pruning was diagnostic only. Pruning and quantization were applied independently; jointly optimizing sparsity and bitwidth, as in (Frantar et al., 2022; Lin et al., 2023), could yield further compression at equal quality. Extending the curriculum with additional pre-training corpora, or distilling from a larger teacher model (Hinton et al., 2015; Sanh et al., 2019) directly into the pruned, quantized student, are directions for improving quality at fixed memory budget.

References

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

William Falcon and The PyTorch Lightning Team. PyTorch Lightning. GitHub repository, 2019.

Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.

Hypersniper. riddles_v1 dataset. Hugging Face Datasets Hub, 2024.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Yanran Li, Hui Su, Xiaoyu Shen, Wenjie Li, Ziqiang Cao, and Shuzi Niu. DailyDialog: A manually labelled multi-turn dialogue dataset. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (IJCNLP)*, 2017.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. Pruning convolutional neural networks for resource efficient inference. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. Q8BERT: Quantized 8bit BERT. In *Proceedings of the NeurIPS EMC2 Workshop*, 2019.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.